

Machine Learning Project

Insurance Risk Prediction

Course Title: Machine Learning

Course Code: BSMALEA1KU

Students:

Student 1: Andrei Sheremet

`andsh@itu.dk`

Student 2: Nam Tram

`hotr@itu.dk`

December 22, 2025

1 Introduction

The project focuses on the problem of predicting risk in a car insurance setting. The aim of this report is to present the steps taken to demonstrate our machine learning knowledge by building models capable of reasonably predicting insurance risk.

As an overview, we first performed exploratory data analysis and basic data cleaning. We then implemented two machine learning models from scratch, using only the NumPy library: a Decision Tree and a Feed-Forward Neural Network. The performance of both models was evaluated using mean Poisson deviance, along with several visualization techniques such as risk decile analysis.

Finally, we were allowed to use any model and library of our choice in order to obtain the best possible results and to compare them with the scratch implementations.

2 Data and Problem Setup

The dataset used in this project provides an overview of vehicular insurance policies and their claims history over a one-year period. It consists of approximately 68,000 entries, where each entry corresponds to a single insurance policy. Each policy is described by 12 variables, listed below:

- **ID**: Unique identifier representing the insurance contract of a driver and their vehicle.
- **ClaimNb**: Number of claims that occurred during the exposure period for the policy.
- **Exposure**: Time at risk in years, ranging from 0 to 1.
- **VehBrand**: Anonymized vehicle brand or brand category.
- **VehGas**: Fuel type of the vehicle.
- **VehPower**: Power rating of the vehicle; higher values correspond to more powerful vehicles.
- **VehAge**: Age of the vehicle in years.

- **DriverAge**: Age of the driver in years.
- **Area**: Density class of the area where the driver resides. This is an ordinal variable starting from “A”, where “A” represents the most rural areas, followed by “B”, “C”, and so on.
- **Region**: Administrative region in which the policyholder resides.
- **Density**: Number of inhabitants per square meter.
- **BonusMalus**: Indicator of the driver’s past claim history. Values below 100 indicate a good past record, while values above 100 correspond to a worse past record.

The dataset is split into training and testing subsets using an 80/20 ratio, with 80% of the data used for model training and 20% reserved for evaluation.

Categorical variables such as vehicle brand, fuel type, and region were encoded using one-hot encoding. The area variable was treated as ordinal and mapped to an integer scale reflecting urban density. No aggressive feature selection was performed, as tree-based and neural models (Look up Section 4) can naturally handle high-dimensional inputs.

Initially, risk was defined as

$$Risk = \frac{ClaimNb}{Exposure},$$

which can be interpreted as the expected number of claims per year. However, at a later stage of the project, we switched to modeling the claim count (*ClaimNb*) directly, as the chosen evaluation metric is more suitable for count-based targets. This choice will be discussed in more detail in a later section.

3 Exploratory Data Analysis

We began working with the dataset by performing basic data validation checks to ensure its adequacy for modeling. In particular, we examined the following aspects:

- Presence of missing (NaN) values

- Presence of duplicate entries
- Validity of feature ranges (e.g., Exposure should lie between 0 and 1, while VehAge, DriverAge, and VehPower should not take negative values)

As a result of these checks, one anomaly was identified: 1,224 entries had exposure values greater than 1, which contradicts the definition of exposure within the dataset. These entries were therefore removed from the data as noise.

Next, we performed a series of univariate visualizations to examine the distributions of individual features. At this stage, the most notable observation was the highly imbalanced distribution of the claim count variable. As shown in Figure 1, the vast majority of policies have zero claims.

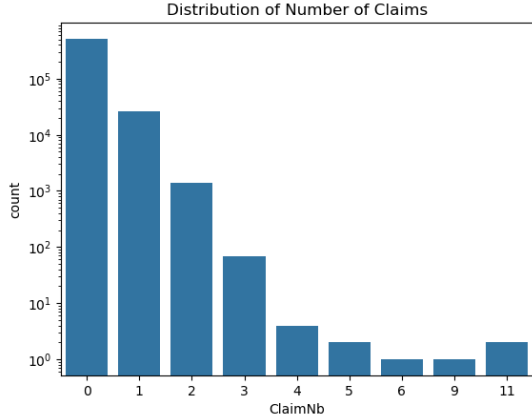


Figure 1: Distribution of ClaimNb

We also computed a correlation matrix between selected numerical features and the target variable, Risk, which is shown in Figure 2. The results indicate that none of the features exhibit a strong linear correlation with the target. This suggests that linear models such as linear regression are unlikely to perform well for this problem and motivates the use of models capable of capturing non-linear relationships and feature interactions.

To further investigate potential patterns associated with higher risk, we discretized several numerical features into bins and examined the mean risk within each bin. This analysis revealed several notable insights. For instance, one anonymized vehicle brand (B12) exhibits a substantially higher average risk (Figure 3), urban areas (Area F) tend to have both higher

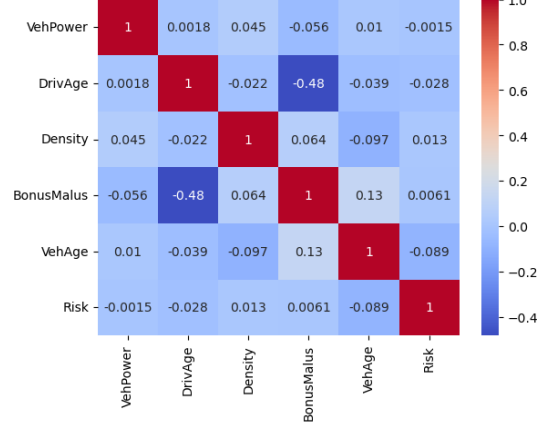


Figure 2: Correlation matrix

risk and greater variance (Figure 5), and risk increases sharply for policies with a Bonus-Malus value above 200 (Figure 4).

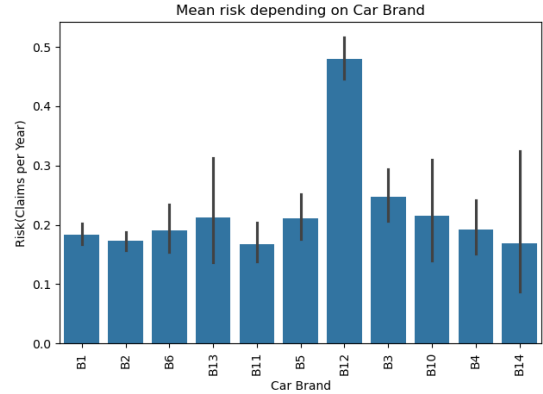


Figure 3: Average risk by vehicle brand

These findings were noted as potential candidates for feature engineering at later stages of the project.

Finally, we examined a two-dimensional PCA projection of the standardized feature space, colored by the logarithm of risk, to assess whether a linear separation between low-risk and high-risk observations exists. As shown in Figure 6, the PCA projection does not reveal a clear separation, indicating that risk is not well explained by linear combinations of the input features.

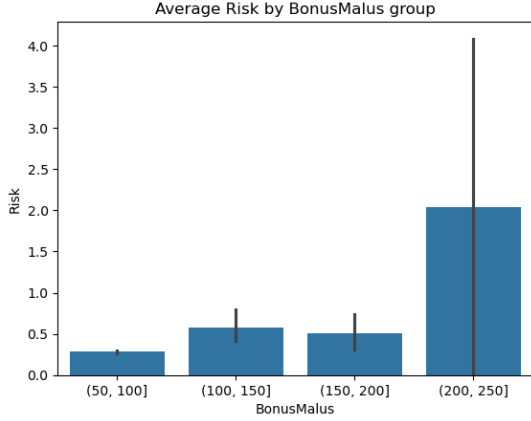


Figure 4: Risk by BonusMalus value

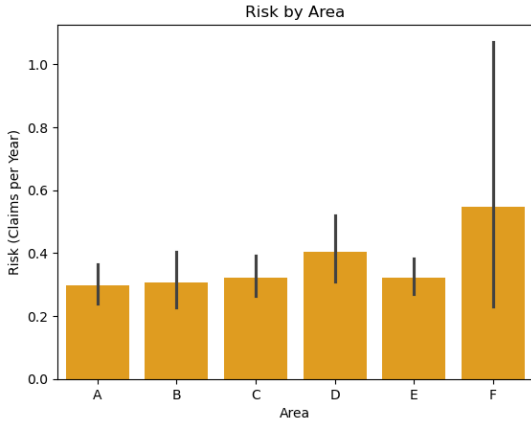


Figure 5: Risk by area type

4 Models

4.1 Decision Tree

The first model we implemented was a Decision Tree. Writing this model from scratch turned out to be relatively intuitive and insightful. The idea behind the algorithm is straightforward: the model iterates over all available features and considers every possible split value for each feature. For every candidate split, it computes the Residual Sum of Squares (RSS) in order to find a split that outputs lowest error in predictions, following the classical CART framework [2].

The split that minimizes RSS is selected as the optimal one. Once the best feature and threshold are found, the dataset is divided into two subsets using a left and a right mask: observations where the feature value is smaller than or equal to the threshold, and observations where it is larger. This procedure is ap-

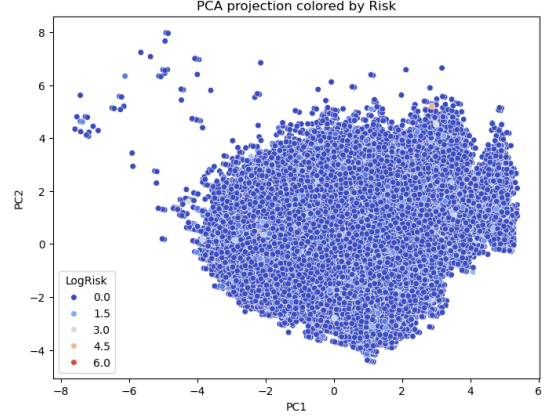


Figure 6: PCA projection colored by log-transformed risk

plied recursively to each subset.

The tree keeps growing until a stopping condition is reached. In our implementation, the growth stops when the maximum tree depth is reached (set to 6 in order to limit overfitting) or when the number of samples in a node becomes too small. When the algorithm reaches a terminal node (a leaf), it gives a mean value of a target feature array, which corresponds to the standard regression tree formulation [2].

Figure 7 illustrates a portion of the learned tree structure, showing the sequence of feature thresholds used for splitting. Figure 8 presents the features that were used most frequently for splitting, in descending order. The most important features according to this criterion are *VehPower*, *DriverAge*, *Density*, *BonusMalus*, and *VehAge*.

4.2 Feed-Forward Neural Network

The second model implemented from scratch was a Feed-Forward Neural Network (FFNN). Compared to the decision tree, this model is considerably more abstract and conceptually complex. The network consists of an input layer, one or more hidden layers, and a single output layer. Each neuron performs a weighted sum of its inputs, where the weights are initially assigned random values, followed by the application of an activation function. The resulting output is then passed to the neurons in the subsequent layer, and this process is repeated until the final prediction is produced by the output layer.

Non-linear activation functions are used to

```

if VehAge <= 0:
    if VehGas_Regular <= False:
        if Region_R54 <= False:
            if VehPower <= 7:
                if VehPower <= 5:
                    if VehBrand_B3 <= False:
                        Predict Risk = 0.239
                    else:
                        Predict Risk = 1.259
                else:
                    if VehBrand_B12 <= False:
                        Predict Risk = 0.451
                    else:
                        Predict Risk = 1.331
            else:
                if VehPower <= 9:
                    if DrivAge <= 25:
                        Predict Risk = 0.672
                    else:
                        Predict Risk = 0.125
                else:
                    if VehPower <= 10:
                        Predict Risk = 1.654
                    else:
                        Predict Risk = 0.315
            ...
        else:
            Predict Risk = 1.084
    else:
        Predict Risk = 2.057

```

Figure 7: A decision tree structure

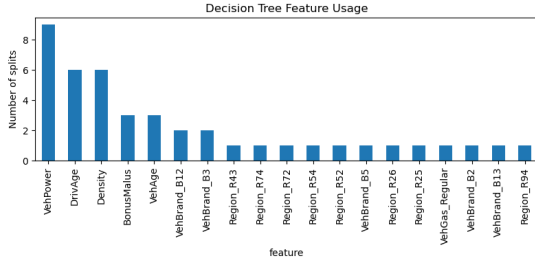


Figure 8: Most used features in the tree

enable the network to model complex relationships between features. In the hidden layers, a Leaky ReLU activation function is applied to introduce non-linearity while mitigating the problem of dead neurons by allowing a small gradient for negative input values.[5] The Leaky ReLU function is defined as

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \leq 0, \end{cases}$$

where α is a small positive constant.

The output layer uses a softplus activation function, which guarantees strictly positive predictions.[1] This property is particu-

larly important when modeling insurance claim counts, as the predicted values must be non-negative. The derivative of the softplus function is given by

$$\frac{d}{dx} \text{Softplus}(x) = \frac{1}{1 + e^{-x}}.$$

After producing a prediction, the model evaluates how far the prediction deviates from the true target value using a Poisson loss function (discussed further in Section 5). The resulting error is then propagated backward through the network using the chain rule, allowing the computation of gradients of the loss function with respect to the model parameters [1]. These gradients indicate how the weights should be adjusted, and the parameters are updated iteratively in the direction that reduces the loss.

4.3 XGBoost

The final model implemented in this project is an ensemble method known as XGBoost. XGBoost is an optimized implementation of gradient boosting, where multiple weak learners are combined to form a strong predictive model.[3] In this case, each weak learner corresponds to a decision tree.

The model is trained sequentially, with each new tree learning to correct the residual errors of the ensemble constructed so far by optimizing the gradient of the loss function. As a result, successive trees focus on observations that are more difficult to predict. The final prediction is obtained as the weighted sum of the contributions from all individual trees.

Unlike approaches where the target variable is defined as claim frequency ($\text{Risk} = \text{ClaimNb} / \text{Exposure}$), this model directly predicts the raw number of claims (ClaimNb). This choice aligns with the `count:poisson` objective used in XGBoost, which is specifically designed for count data. Instead of dividing by exposure, the Exposure variable is incorporated as a sample weight during training. This allows the model to learn expected claim counts while properly scaling each policy's contribution. This approach preserves the discrete nature of claims and ensures compatibility with Poisson deviance as the evaluation metric.

For the hyperparameter configuration shown in Figure 9, the number of trees was set to 400,

```

xgb = xgb.XGBRegressor(
    objective="count:poisson",
    n_estimators=400,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8
)

xgb.fit(X_train, y_train, sample_weight=w)
y_pred=xgb.predict(X_test)

```

Figure 9: XGBoost Modelling

allowing the model to capture complex patterns in the data. A moderate learning rate was used to ensure stable training and to reduce the risk of overfitting by gradually incorporating each tree’s contribution. To further control model complexity, the maximum depth of individual trees was limited to 6. This depth allows the model to capture feature interactions while preventing excessive memorization of noise. Additionally, row and column subsampling of 80% were applied to improve generalization.

To verify the correctness of the model implementation, several diagnostic plots were used. One such method is a decile plot based on predicted claim counts. Policies are grouped into ten buckets according to their predicted risk, from lowest to highest. For each decile, the average observed claim count is computed.

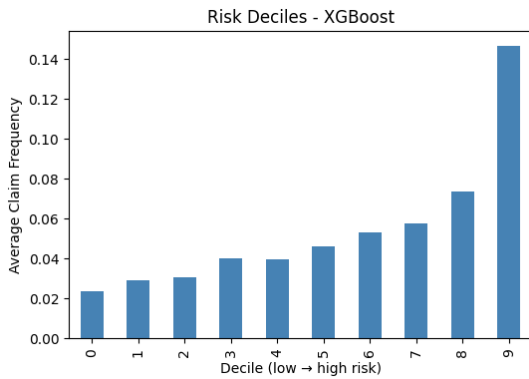


Figure 10: Risk decile analysis for XGBoost

As shown in Figure 10, the average observed claims increase monotonically from the lowest to the highest decile. This confirms that the model correctly ranks policies by risk, concentrating higher claim frequencies in higher predicted risk groups.

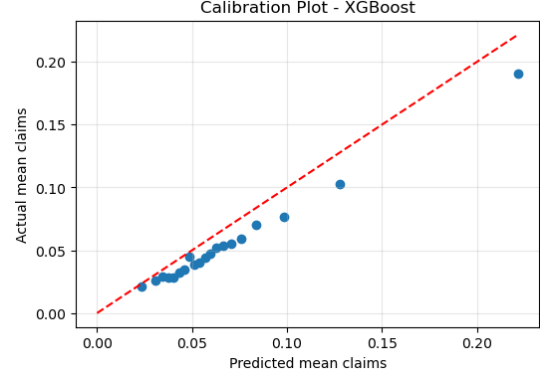


Figure 11: Calibration plot for XGBoost

A calibration plot was also used to assess numerical accuracy. Predicted claim counts were binned, and the average predicted and observed claims were computed for each bin. As shown in Figure 11, the points lie close to the diagonal, indicating good agreement between predicted and observed claim counts. This demonstrates that the model not only ranks policies correctly but also produces predictions that are numerically consistent with the data.

5 Evaluation Metrics

5.1 Mean Poisson Deviance

To evaluate the numerical accuracy of predicted claim counts, Mean Poisson Deviance (MPD) was used. Mean Poisson Deviance is derived from generalized linear models for count data [6]. It is defined as

$$\text{MPD} = \frac{1}{n} \sum_{i=1}^n 2 \left[y_i \log \left(\frac{y_i}{\hat{y}_i} \right) - (y_i - \hat{y}_i) \right].$$

Mean Poisson Deviance measures the discrepancy between predicted and observed claim counts. Lower MPD values indicate better model fit. This metric is particularly suitable for count data and aligns with the Poisson objective used in the XGBoost model, making it an appropriate choice for comparison across all implemented models.

Poisson deviance is particularly suitable in an insurance context, as it penalizes underestimation of high claim counts more strongly, which aligns with the insurer’s risk management objectives.

5.2 Lift Score

To assess the model’s ability to identify high-risk policies, the Lift Score was used. The lift compares the average observed claim count in the highest predicted risk decile to the overall average claim count and is defined as

$$\text{Lift} = \frac{\text{Average observed claims in top decile}}{\text{Overall average claims}}.$$

A lift value greater than 1 indicates that the model successfully concentrates claims within the highest predicted risk group, which is particularly important for highly imbalanced insurance datasets.

5.3 Precision, Recall

Given the highly imbalanced nature of the dataset, additional evaluation was performed using Precision–Recall analysis. These metrics focus on the minority class—policies with at least one claim—and are therefore more informative than accuracy-based measures in this context.

Precision measures the proportion of predicted claimers that are correct, while recall measures the proportion of actual claimers that are successfully identified. Their definitions are given by

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

The Precision–Recall curve illustrates the trade-off between these two quantities across different classification thresholds. The Average Precision score summarizes the area under this curve and provides a single metric reflecting the model’s ability to detect claimers in an imbalanced setting.

6 Results

6.1 XGBoost

The XGBoost model demonstrates the strongest overall performance across all evaluation metrics. It achieves the lowest Mean Poisson Deviance of 0.307, indicating accurate prediction of claim counts. The top-decile

lift of 2.726 shows that the model effectively concentrates risk, with the highest predicted decile exhibiting more than twice the average claim frequency.

The decile plot in Figure 10 confirms a clear monotonic increase in observed claims across deciles, validating correct risk ranking. Furthermore, the calibration plot in Figure 11 shows strong agreement between predicted and observed claim counts. The Precision–Recall curve in Figure 12, together with an Average Precision score of 0.115, indicates that the model performs well in identifying claimers despite the highly imbalanced nature of the dataset.

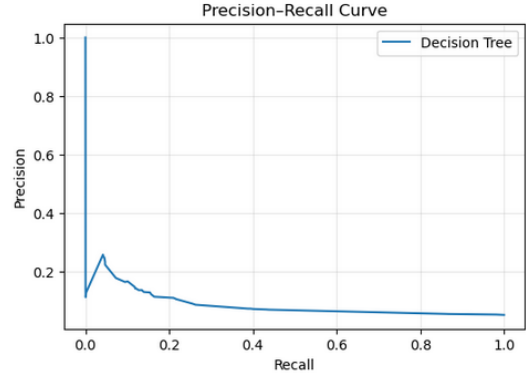


Figure 12: Precision-Recall Curve XGBoost

6.2 Decision Tree

The Decision Tree model achieves a higher Mean Poisson Deviance of approximately 0.358, indicating weaker numerical accuracy compared to XGBoost. However, the model still performs well in terms of risk ranking.

As shown in the decile plot in Figure 14, the average observed claim count increases consistently across predicted risk deciles. The top-decile lift of 2.427 demonstrates that the Decision Tree is capable of effectively segmenting policies into distinct risk groups.

The calibration plot in Figure 15 reveals a more polarized relationship between predicted and observed claim counts, suggesting less stable numerical predictions. This behavior is also reflected in the Precision–Recall curve in Figure 13, which shows reasonable performance at lower recall levels but degrades as recall increases.

In summary, while the Decision Tree does not achieve the same level of numerical accuracy as XGBoost, it remains effective for risk segmentation and offers the advantage of interpretability.

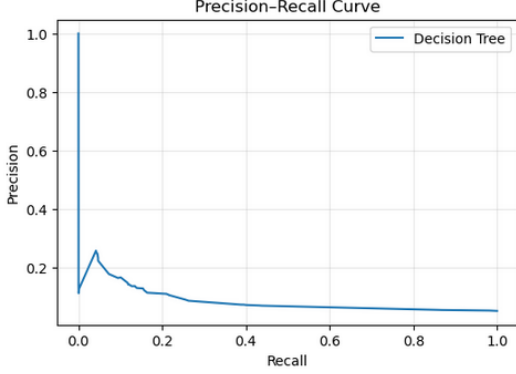


Figure 13: Precision-Recall Curve Decision Tree

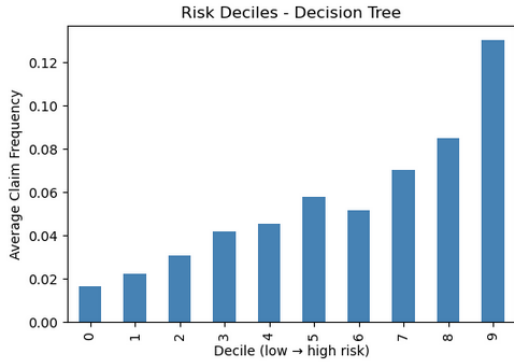


Figure 14: Decile Graph Decision Tree

6.3 Feed-Forward Neural Network

The Feed-Forward Neural Network implemented from scratch achieves a Mean Poisson Deviance of 0.341, indicating a reasonable overall fit to the data. However, its ranking performance is substantially weaker compared to the tree-based models.

The decile analysis in Figure 17 shows little separation between predicted risk groups, with a top-decile lift close to 1. This suggests that the network struggles to distinguish high-risk policies from the rest of the population.

The calibration plot in Figure 18 further indicates that the model tends to produce predictions close to the global mean, resulting in limited variation across policies. This behavior

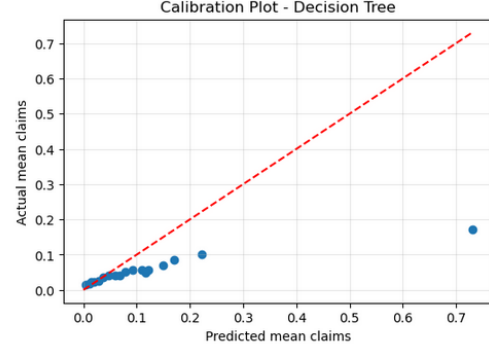


Figure 15: Calibration Plot Decision Tree

is consistent with the Precision-Recall curve in Figure 16, which shows weak discrimination ability and a low Average Precision score.

These results suggest that, while the FFNN provides acceptable average predictions under a likelihood-based loss, it is not well suited for risk ranking in this setting. This limitation is likely due to the simplicity of the network architecture and the absence of optimization mechanisms specifically targeting ranking performance.

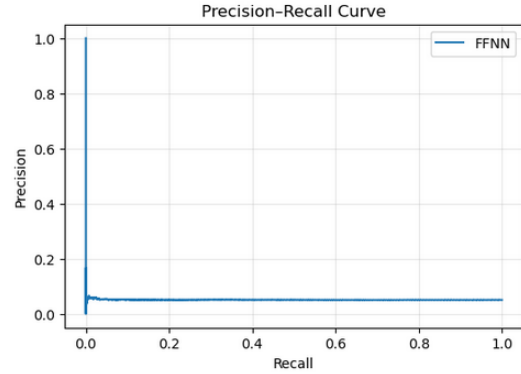


Figure 16: Precision-Recall FFNN

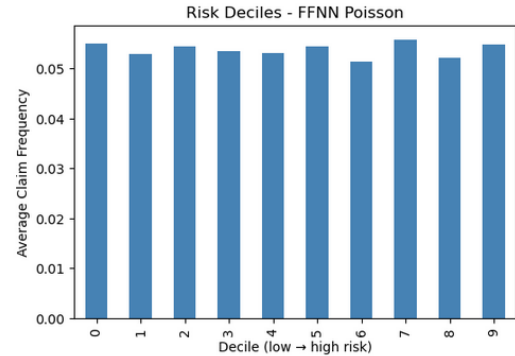


Figure 17: Decile Graph FFNN

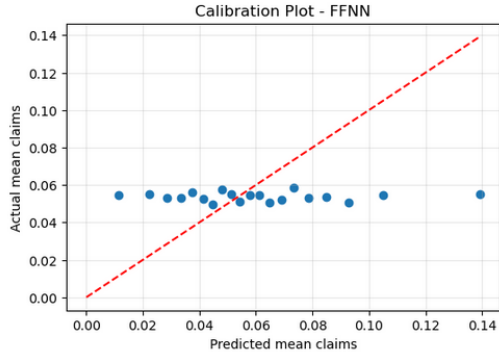


Figure 18: Calibration Plot FFNN

7 Discussion

The results of this project demonstrate the challenges of insurance risk prediction on highly imbalanced count data. Since the majority of policies have zero claims, traditional regression metrics were not suitable. Mean Poisson Deviance was therefore chosen, as it is designed for count data and better reflects prediction quality in this setting.

Among the three models, XGBoost achieved the best overall performance. It obtained the lowest Poisson deviance and showed strong separation between low-risk and high-risk policies in the decile analysis. The monotonic increase of observed claims across deciles and the well-aligned calibration plot indicate that the model ranks policies correctly and produces numerically reliable predictions.

The decision tree model showed moderate performance. While its Poisson deviance was higher than that of XGBoost, the decile plot and lift score indicate that it can still distinguish higher-risk policies from lower-risk ones. However, its calibration was weaker, suggesting limited accuracy in estimating claim intensity. This is expected for a single tree, which has restricted capacity to model complex feature interactions.

The custom feed-forward neural network achieved a Poisson deviance comparable to the decision tree but performed poorly on ranking-based metrics such as lift and decile plots. The calibration analysis shows that the network tends to predict values close to the global mean, indicating underfitting. This behavior likely results from both the simplicity of the

network architecture and the difficulty neural networks face on tabular data without extensive tuning and feature engineering.[4]

Overall, these results highlight that Poisson deviance alone is not sufficient to assess model usefulness. Ranking-based evaluations such as deciles and lift are crucial in an insurance context. XGBoost consistently outperformed the other models, confirming the suitability of tree-based ensemble methods for structured, imbalanced insurance data.

8 Conclusion

In this project, we implemented a decision tree regressor and a feed-forward neural network from scratch and compared them with an off-the-shelf XGBoost model. Because the target is claim counts with a large mass at zero, we evaluated models using Mean Poisson Deviance and ranking-based diagnostics (decile plot and lift). Overall, XGBoost achieved the best performance, showing both lower Poisson deviance and stronger separation of low- vs high-risk policies. The decision tree provided interpretable splits and reasonable ranking, but was less accurate and less well-calibrated. The custom FFNN improved in Poisson deviance after tuning, but still struggled to rank high-risk policies, suggesting underfitting and limited optimization in the from-scratch implementation. The results indicate that boosted trees are a strong choice for claim frequency modeling on tabular insurance data, while neural networks require more advanced training setup to be competitive.

References

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- [3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *arXiv preprint arXiv:1603.02754*, 2016.
- [4] Leo Grinsztajn, Edouard Oyallon, and Gael Varoquaux. Why do tree-based mod-

els still outperform deep learning on tabular data? *NeurIPS*, 2022.

- [5] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. Rectifier nonlinearities improve neural network acoustic models. *ICML Workshop*, 2013.
- [6] Peter McCullagh and John Nelder. Generalized linear models. *Chapman and Hall*, 1989.
- [7] OpenAI. Assistance in model implementation and explanation, 2025. ChatGPT (GPT-4) was used for conceptual explanations and debugging assistance.

During the implementation of the from-scratch models, conceptual clarification and debugging assistance were supported using an AI-based language model [7].